

1 Description of the Proposed Solution

Clinical codification represents a critical administrative and clinical bottleneck in healthcare systems, requiring the translation of free-text diagnostic and procedural notes into standardized classification codes. In this work, we address the single-label multiclass clinical codification task, which maps raw, brief Spanish clinical report literals into their corresponding first-character ICD-10 categories, spanning 36 distinct classes (0–9 and A–Z).

Our final engineering pipeline is designed as a highly specialized, sequential deep learning pipeline optimized to handle severe class imbalance, Catalan-Spanish bilingual expressions, medical jargon, and hardware VRAM limitations. The core components of our proposed architecture and text-processing workflow are detailed below.

1.1 Sequential Pipeline Architecture

The processing of a raw clinical literal flows through a multi-stage sequential architecture:

1. **Text Preprocessing:** The raw clinical report is cleaned using a lightweight preprocessing filter implemented in the `preprocess_text` function within our pipeline. This step casts the inputs to strings, strips leading and trailing whitespaces, and collapses multiple consecutive whitespaces into a single space:

$$x_{\text{clean}} = \text{RegexSub}(\text{Strip}(\text{String}(x_{\text{raw}})), '\s+', ' ') \quad (1)$$

Crucially, we do not perform lowercase normalization, accent removal, or punctuation stripping. Preserving original casing and accents is essential, as the downstream specialized clinical tokenizer contains unique subword representations for these clinical forms.

2. **Subword Tokenization:** The cleaned text is tokenized using the pre-trained Spanish biomedical tokenizer associated with the '*PlanTL-GOB-ES/roberta-base-biomedical-clinical-es*' backbone. The sequences are padded or truncated to a highly optimized maximum length of 64 tokens. While standard pipelines use a max length of 128, our analysis of the dataset made us discover that the longest clinical literal in `codification_data.csv` contains 49 subwords. Setting the limit to 64 reduces RAM consumption and accelerates training speeds with zero loss of clinical information.
3. **Biomedical Transformer Encoder:** The tokenized IDs are passed through the 12-layer Spanish biomedical-clinical RoBERTa model. Standard multilingual models are trained on general web corpora and struggle to encode Spanish medical abbreviations (e.g. VHC, HPB, DG) and diagnostic idioms. This backbone, pre-trained on over 1B tokens of Spanish clinical notes, and medical publications, provides robust pre-built clinical contextual embeddings of dimension $d_{\text{odel}} = 768$.

4. **Custom Attention Pooling:** To aggregate the sequence hidden states into a single sequence vector, we replace standard $\langle \text{CLS} \rangle$ and Mean pooling with a parameterized **Attention Pooling Layer**. Standard $\langle \text{CLS} \rangle$ pooling suffers because the sequence-start token in RoBERTa is trained on general masked language modeling and fails to focus on short, highly dense medical nouns. Mean pooling assigns equal weight to non-informative terms (e.g. Catalan/Spanish prepositions like **de**, **en**).
5. **Multi-Sample Dropout Regularization:** To prevent overfitting on small clinical target sets, the pooled representation $\mathbf{v}$ is processed by a Multi-Sample Dropout head. We initialize five parallel dropout paths with a dropout probability of $p = 0.15$. During training, the representation is cloned five times, passed through independent dropout masks, and fed into the shared linear classification head.

Averaging the logits acts as a robust ensemble regularizer, smoothing the decision boundaries and preventing the classifier head from co-adapting to specific high-frequency clinical words.
6. **Linear Classification Head:** A linear projection layer maps the averaged 768-dimensional vector to the 36 logits mentioned, corresponding to the ICD-10 clinical categories.

1.2 Justification of the Deep Learning approach

Our group transition from traditional statistical methods to a deep clinical transformer architecture was due to the challenges inherent to medical narratives:

- **Failure of Traditional Term Frequency Models:** Traditional statistical baselines rely on exact term matching. Clinical reports frequently use non-standard abbreviations, misspellings, multi-word clinical concepts... For example, TF-IDF is incapable of mapping the abbreviation HTA to *Hipertensión Arterial* or VHC to *Hepatitis C*, treating them as entirely different features.
- **Bilingual Medical Terminology:** The dataset is characterized by a fluid, real-world mix of Spanish and Catalan terminology. A deep learning model trained on localized clinical text successfully maps these distinct lexical words to the same semantic space, but statistical methods would have required manually bilingual synonym dictionaries.

2 Experiments and Evaluation Setup

To assess our models and prevent overestimating performance, we established a robust validation partition and systematic training setup that clones real clinical deployment constraints.

2.1 Dataset Partitioning and the "Split-Before-Augment" Protocol

The primary clinical dataset (`codification_data.csv`) consists of 13,702 short clinical report literals. The classification target spans 36 classes. Ten classes correspond to the **ICD-10 Procedure Coding System**, where digit 0 indicates surgical procedures and digit 3 indicates administrative procedures. The remaining 26 classes correspond to the **ICD-10 Clinical Modification (ICD-10-CM)** diagnostic categories. We implemented a **stratified 80/20 train/validation split** using the scikit-learn `train_test_split` utility, preserving class proportions across both sets:

- **Stratified Training Split:** 10,961 samples.
- **Clean Validation Split:** 2,741 samples.

2.2 Hyperparameter Configuration

For both the baseline and advanced models, we systematically tracked and tuned several hyperparameters. Table 1 outlines the exact experimental configurations used during training.

2.3 Evaluation Metrics and the Insufficiency of Accuracy

Model evaluation was performed using Loss curves, Precision, Recall, and the Macro F1-Score. In our clinical codification task, Accuracy alone is a highly misleading and insufficient metric.

Because the primary hospital data is imbalanced by some factors, a naive model that predicts only these three categories would mathematically achieve over 60% validation accuracy while failing to classify the other 33 clinical classes. In a clinical setting, errors like those would have catastrophic implications for patients.

Therefore, we used the **Macro-averaged F1-Score** as our primary optimization and early stopping target:

Table 1: Hyperparameter Configurations for Baseline and Advanced Models

Hyperparameter	Baseline Model	Advanced Optimized Model
Pre-trained Backbone	roberta-base-biomedical-clinical-es	roberta-base-biomedical-clinical-es
Sequence Max Length (L)	128	64
Optimizer	AdamW	AdamW
Learning Rate (Backbone)	2.0×10^{-5}	2.0×10^{-5}
Learning Rate (Heads)	2.0×10^{-5}	2.0×10^{-5}
Weight Decay	0.01	0.01
Warmup Ratio	10%	10%
Physical Batch Size	128	32
Gradient Accumulation Steps	None	4
Effective Batch Size	128	128
Max Training Epochs	50	50
Early Stopping Metric	Validation Accuracy	Validation Macro F1-Score
Early Stopping Patience	10 epochs	10 epochs
Encoder Frozen Layers	None (All 12 active)	Embeddings + Bottom 6 Layers
Dropout Probability (p)	0.10	0.15 (5-sample Multi-Dropout)
Loss Criterion	Standard Cross-Entropy	Balanced Class-Weighted Cross-Entropy

3 Critical Analysis of Results

Following the implementation of our deep learning models, we made a comparative analysis of the validation and test results. Table 2 breakdown of model performance.

Table 2: Model Performance Comparison on Clinical Codification (Stratified Split)

Pipeline Architecture	Val. Loss	Val. Acc.	Val. Macro F1	Kaggle Test Acc.
Baseline RoBERTa + (CLS) Pooling	1.724	0.565	0.245	0.558
Baseline RoBERTa + Mean Pooling	1.682	0.570	0.258	0.562
RoBERTa + Attention Pool + Multi-Dropout (No Aug.)	1.104	0.685	0.452	0.678
Our Final Optimized Pipeline (Aug. + Weights + Freeze)	0.682	0.804	0.718	0.792

3.1 Successes and Shortcomings of the Advanced Model

Where the Advanced Model Succeeded:

- **Generalization to Rare Diagnoses:** The combination of balanced class weights and targeted dictionary augmentation enabled the model to correctly identify rare diagnostic classes.
- **Disambiguating Procedural vs. Diagnostic Contexts:** The model successfully distinguished between diagnostic records and their corresponding clinical procedures.
- **Robustness to Lexical Noise:** Attention pooling allowed the classification head to bypass syntactic noise and focus on key clinical terms.

Where the Advanced Model didn't succeed:

- **Ambiguity in Ultra-Short Literals:** Extremely brief literals consisting of a single abbreviation remained a major bottleneck. For example, the literal **ia** can represent *Insuficiencia Aórtica* or *Inteligencia Artificial* or *Inseminación Artificial*. The lack of context in some sentences made the model defaulted to the maternally-biased class 0 or procedure class 0, leading to systematic errors.
- **Granular boundary confusion:** The model occasionally struggled with the boundary between specialized endocrinology and general metabolic symptoms. Literals like **Deshidratación hipernatrémica** were misclassified as general symptoms instead of metabolic disorders .

4 Conclusions and Future Work

In this academic project, our group successfully designed, optimized, and deployed an end-to-end clinical text classification pipeline for ICD-10 codification. By leveraging a specialized Spanish biomedical pre-trained backbone, designing a custom attention pooling mechanism, and implementing a strict stratified split-before-augment strategy, we transformed a difficult baseline into a highly robust clinical model capable of generalizing to rare diagnostic and procedural categories.

4.1 Collaborative and Engineering Takeaways

This project taught us that brute-force scaling of deep learning models is sub-optimal when working with real-world medical data and restricted academic hardware. The key engineering takeaways from our collaboration are:

- Data quality and separation are essential: setting up a clean, leak-free validation split before running dictionary augmentations was the single most important factor in obtaining reliable evaluation metrics.
- Custom pooling and regularization, like Multi-Sample Dropout, provide substantial performance gains over simple pooling baselines, proving that custom network heads are vital when fine-tuning general encoders on domain-specific, short literals.
- Proactive hardware optimization is an essential skill in AI engineering, allowing high-quality research to progress even under conditions like the ones we met in this project.

4.2 Sociotechnical and Ethical Implications

Deploying machine learning models in clinical environments carries significant ethical responsibility. For instance, the bias and underrepresentations lead our

training data to be heavily biased toward common hospital encounters. A model trained on this data might underperform on marginalized demographics or rare medical conditions, potentially leading to administrative misclassifications.